

Module 13 Lab:

Secure SDLC and OWASP Top 10

JR Clark (002035589)

NEU DAMG7350 (Spring 2026)

Introduction

Modern web applications are frequently targeted through vulnerabilities created by insecure coding practices. The OWASP Top 10 provides a practical framework for understanding these risks and the types of weaknesses that appear when user input is not validated, sanitized, or properly controlled. This lab focused on reproducing three common vulnerabilities -- SQL Injection, Cross-Site Scripting (XSS), and Broken Access Control -- using small Flask applications. All implementation steps and screenshots referenced in this report are documented in the Appendix.

Vulnerability 1: Injection Attacks (A03:2021, A05:2021):

SQL Injection arises when an application constructs SQL statements by concatenating user input directly into the query string. This violates a core Secure SDLC principle: untrusted input must never influence backend command structure. In the vulnerable version of `vulnerable_login.py`, the f-string allowed the attacker to terminate the intended query and inject SQL comment syntax. The payload worked because the database parsed the input as SQL grammar rather than as a literal value, enabling an authentication bypass.

To eliminate the vulnerability, I replaced the string-formatted query with a parameterized SQL statement. Parameterized queries bind user input as data rather than executable SQL, preventing attackers from reshaping the query structure. After applying the fix, the same payload no longer succeeded, confirming that the root cause—not just the symptom—had been addressed.

SQL Injection remains one of the most severe web vulnerabilities because it can expose or modify sensitive data and compromise authentication logic. OWASP identifies parameterized

queries as the most reliable defense, supported by least-privilege database accounts and centralized query handling to reduce the risk of injection across the application.

Injection attacks are dangerous because they can compromise both backend systems as well as other connected clients and result in completely compromising or destroying a system.

Vulnerability 2: Cross-Site Scripting (XSS) (A01:2025)

The XSS vulnerability resulted from directly embedding untrusted input into an HTML response. This breaks the SDLC requirement for output encoding and allows user-controlled data to become executable script. Because the application inserted the `name` parameter directly into the HTML, the browser interpreted the injected `<script>` tag as JavaScript, demonstrating a reflected XSS attack.

To mitigate the issue, I used the `escape()` function from the `markupsafe` library to encode special characters before rendering them in the browser. This ensures that characters such as `<` and `>` are treated as text rather than script delimiters. After applying the fix, the same payload rendered harmlessly as text, confirming that output encoding neutralized the vulnerability.

XSS persists in modern applications because browsers inherently trust server-supplied HTML. OWASP recommends context-aware output encoding as the primary defense, supported by input validation and safe templating practices. These controls ensure that user input cannot be interpreted as active content regardless of how it is delivered.

OWASP describes the variety of attacks based on XSS as “limitless” and typically involve exfiltration of user data or posting unregulated material on a trusted website. Some examples include “allowing an attacker to modify a press release or news item could affect a

company's stock price or lessen consumer confidence" or have serious medical impact if an attacker can modify dosage information of medication (OWASP, n.d. (a)).

Vulnerability 3: Broken Access Control (OWASP A01:2021)

Broken Access Control occurs when an application exposes sensitive functionality without verifying the identity or privileges of the requester. In the vulnerable version of `broken_access.py`, the `/admin` route implicitly trusted all clients, violating the SDLC principle of least privilege. Simply visiting the URL in a browser was enough to retrieve confidential data, demonstrating that the application performed no server-side authorization checks.

This vulnerability highlights the critical distinction between authentication (verifying who a user is) and authorization (verifying what a user is allowed to do). Even if authentication is implemented correctly, failing to enforce authorization allows attackers to bypass privilege boundaries. Broken Access Control is consistently ranked by OWASP as the top web application risk because it appears in many forms, including Insecure Direct Object References (IDOR), where attackers manipulate identifiers to access other users' data.

The vulnerable `/admin` endpoint in this lab could be exploited through direct navigation, automated scanning, or manipulation of cookies or headers to impersonate privileged users. These exploitation paths illustrate why access control must be enforced server-side rather than relying on obscurity or client-controlled mechanisms.

Appendix A Step3 17–21 contains the screenshots demonstrating the vulnerable `/admin` access, the corrected authorization check, and the resulting 403 Forbidden response.

Production systems require more robust controls aligned with OWASP recommendations, including: role-based access control (RBAC), session-based authentication, framework-level

authorization, least privilege enforcement, and centralized access control logic. In real-world systems, such vulnerabilities can lead to large-scale data breaches, unauthorized disclosure of sensitive records, and regulatory compliance violations.

Conclusion:

These three demonstrations highlight how small implementation decisions can create significant security risks when secure coding practices are not embedded into the development lifecycle. SQL Injection, XSS, and Broken Access Control all stem from the same underlying issue: trusting user input without validation, encoding, or authorization checks. The fixes applied in this lab—parameterized queries, output escaping, and explicit access control—map directly to Secure SDLC activities such as secure design reviews, code analysis, and security testing.

By integrating these controls early and consistently, development teams can prevent entire classes of vulnerabilities rather than reacting to them after deployment. Secure coding is not a separate activity but an essential part of building reliable, trustworthy software.

References

- OWASP Foundation. (n.d.(a)). Cross-site scripting (XSS). <https://owasp.org/www-community/attacks/xss/>
- OWASP Foundation. (n.d.(b)). Cross-site scripting prevention cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP Foundation. (n.d.(c)). Injection prevention cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Injection_Prevention_Cheat_Sheet.html
- OWASP Foundation. (2021). A03:2021 – Injection. https://owasp.org/Top10/2021/A03_2021-Injection/
- OWASP Foundation. (n.d.(d)). Injection flaws. https://owasp.org/www-community/Injection_Flaws
- OWASP Foundation. (2021). *OWASP Top Ten Web Application Security Risks*. <https://owasp.org/www-project-top-ten/>
- OWASP Foundation. (2025). A01:2025 – Broken access control. https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/
- OWASP Foundation. (2025). A05:2025 – Injection. https://owasp.org/Top10/2025/A05_2025-Injection/
- Sawant, A. (2026). Secure SDLC and OWASP Top 10 [Online course]. Coursera & Northeastern University.

Methodology and Use of AI Tools

I completed all technical steps of this lab directly in the Kali Linux virtual machine. All screenshots and observations were generated from my own live capture sessions. I used Microsoft Copilot to create the broken access python script and as a writing and organizational aid to help structure the report, refine transitions between sections, and ensure alignment with the assignment rubric. I wrote up the scenarios and asked CoPilot to draft the configuration file based on my specifications. All analysis, interpretations, and conclusions reflect my own understanding of the material and the concepts covered in Module 13.

APPENDIX A: Documented Process

The following appendix provides a complete, chronological record of the technical work performed for this lab. Each step includes a caption describing the purpose and security relevance of the action, along with a corresponding screenshot that demonstrates the environment setup, vulnerable code execution, exploitation results, and verification of secure fixes. These figures collectively serve as evidence of successful implementation, testing, and mitigation of the three OWASP vulnerabilities analyzed in the main report.

Step 1: Verify Python 3 installation

Confirms the environment supports modern Flask and SQLite libraries required for secure and vulnerable application testing.

```
(jenn@kali)-[~]
$ sudo apt -y install python3 python3-pip sqlite3
[sudo] password for jenn:
python3 is already the newest version (3.13.9-3).
python3 set to manually installed.
python3-pip is already the newest version (26.0.1+dfsg-1).
python3-pip set to manually installed.
sqlite3 is already the newest version (3.46.1-9+b1).
sqlite3 set to manually installed.
The following packages were automatically installed and are no longer require
d:
curlftpfs                                libpostproc58
libaudio2                                librubberband2
libavfilter10                            libsframe2
libavformat61                            libsnmp40t64
libblosc2-4                              libsphinxbase3t64
libconfig-inifiles-perl                  libswscale8
libcrypt-dev                             libtsk19t64
libdlt2                                  libvdpau-va-gl1
libdnnl3.6                               libxnnpack0.20241108
libfuse2t64                              libzxing3
libgav1-1                                mesa-vaapi-drivers
libgdk-pixbuf2.0-bin                     pocketsphinx-en-us
libjpegutils-2.1-0t64                    python3-aiocache
libmpeg2encpp-2.1-0t64                   python3-aiomcache
libmpeg2-2.1-0t64                        python3-fs
libmupdf25.1                             python3-pysnmp4
libonnx1t64                              python3-terminaltables
libonnxruntime-providers                  python3-wapiti-arsenic
libonnxruntime1.21                       python3-yaswfp
libplacebo351                             ruby-unf-ext
libpocketsphinx3                         vdpau-driver-all
Use 'sudo apt autoremove' to remove them.

Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 18
```

Figure 1. Python 3 installation verified

Confirms the environment supports Flask and SQLite for vulnerability testing.

Step 2: Create lab16 directory and initialize virtual environment

Establishes an isolated workspace to prevent dependency conflicts and follows SDLC best practices for controlled development environments.

```
(jenn@kali)-[~]
$ ls -a
.          enum4linux.txt  Pictures
..         .face          .profile
16         .face.icon    Public
'Bandit Game' .gnupg             .ssh
.bash_logout gobuster.txt       .sudo_as_admin_successful
.bashrc     .ICEauthority      Templates
.bashrc.original .java             Videos
.cache      lab         .Xauthority
cd          lab16      .xsession-errors
.config     .local             .xsession-errors.old
Desktop     .mozilla           .zprofile
.dmr        Music      .zsh_history
Documents   nmap_full_tcp.txt  .zshrc
Downloads   nmap_svcs.txt

(jenn@kali)-[~]
$ cd lab16

(jenn@kali)-[~/lab16]
$ python3 -m venv .venv

(jenn@kali)-[~/lab16]
$ source .venv/bin/activate

(.venv)-(jenn@kali)-[~/lab16]
```

Figure 2. Virtual environment created and activated

Shows isolated workspace setup following SDLC best practices.

Step 3: Install Flask inside the virtual environment

Ensures the application framework is properly scoped to the project and avoids system-wide package contamination.

```
(.venv)-(jenn@kali)-[~/lab16]
$ pip install flask
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.3.2-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Downloading markupsafe-3.0.3-cp313-cp313-manylinux2014_aarch64.manylinux2_17_aarch64.manylinux2_28_aarch64.whl.metadata (2.7 kB)
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.3.2-py3-none-any.whl (108 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading markupsafe-3.0.3-cp313-cp313-manylinux2014_aarch64.manylinux2_17_aarch64.manylinux2_28_aarch64.whl (24 kB)
Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, flask
Successfully installed blinker-1.9.0 click-8.3.2 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8

(.venv)-(jenn@kali)-[~/lab16]
$
```

Figure 3. Flask installed in virtual environment

Demonstrates controlled dependency installation.

Step 4: Confirm SQLite installation

Validates that the backend database engine is available for demonstrating SQL Injection and secure parameterized queries.

```
(.venv)-(jenn@kali)-[~/lab16]
$ sudo apt -y install sqlite3
sqlite3 is already the newest version (3.46.1-9+b1).
The following packages were automatically installed and are no longer required:
d:
curlftpfs                                libpostproc58
libaudio2                                librubberband2
libavfilter10                            libsframe2
libavformat61                            libsnmp40t64
libblosc2-4                              libspinxbase3t64
libconfig-inifiles-perl                  libswscale8
libcrypt-dev                             libtsk19t64
libdlt2                                  libvdpau-va-gl1
libdnnl3.6                              libxnnpack0.20241108
libfuse2t64                             libzxing3
libgav1-1                               mesa-vdpau-drivers
libgdk-pixbuf2.0-bin                    pocketsphinx-en-us
libmjpegutils-2.1-0t64                  python3-aiocache
libmpeg2encpp-2.1-0t64                  python3-aiomcache
libmpeg2-2.1-0t64                       python3-fs
libmupdf25.1                            python3-pysnmp4
libonnx1t64                             python3-terminaltables
libonnxruntime-providers                 python3-wapiti-arsenic
libonnxruntime1.21                      python3-yaswfp
libplacebo351                            ruby-unf-ext
libpocketsphinx3                         vdpau-driver-all
Use 'sudo apt autoremove' to remove them.

Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 18
```

Figure 4. SQLite installation confirmed

Validates database availability for SQL Injection testing.

Step 5: Create the database table used in the login demo

Sets up the schema required for authentication testing, enabling both the vulnerable and secure SQL query paths.

```
(.venv)-(jenn@kali)-[~/lab16]
$ python3 -c 'PY'
import sqlite3
conn = sqlite3.connect('users.db')
cur = conn.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS users (username TEXT, password TEXT)")
cur.execute("INSERT OR IGNORE INTO users VALUES ('admin','admin123')")
conn.commit()
conn.close()
print("users.db created")
PY
users.db created

(.venv)-(jenn@kali)-[~/lab16]
$
```

Figure 5. Database table created

Shows schema setup for authentication testing.

Step 6: Create vulnerable_login.py

Implements the intentionally insecure SQL query logic needed to demonstrate SQL Injection in a controlled environment.

A screenshot of a terminal window titled "(.venv)jenn@kali: ~/lab16". The window shows the GNU nano 8.7.1 editor with a file named vulnerable_login.py. The code is a Flask application that uses SQLite to store user credentials. It features a login endpoint that concatenates user input into an SQL query, making it vulnerable to SQL injection. The application also includes a main block to initialize a test database with a user named 'admin1' and password 'admin123'.

```
from flask import Flask, request
import sqlite3

app = Flask(__name__)

@app.route('/login', methods=['GET'])
def login():
    username = request.args.get('username', '')
    password = request.args.get('password', '')

    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    # VULNERABLE QUERY (string concatenation)
    query = f"SELECT * FROM users WHERE username='{username}' AND password='{password}'"
    cursor.execute(query)
    user = cursor.fetchone()

    conn.close()

    if user:
        return f>Welcome, {username}!"
    else:
        return "Invalid credentials"

if __name__ == '__main__':
    # Initialize a small test DB
    conn = sqlite3.connect('users.db')
    conn.execute("CREATE TABLE IF NOT EXISTS users (username TEXT, password TEXT)")
    conn.execute("INSERT OR IGNORE INTO users VALUES ('admin1', 'admin123')")
    conn.commit()
    conn.close()

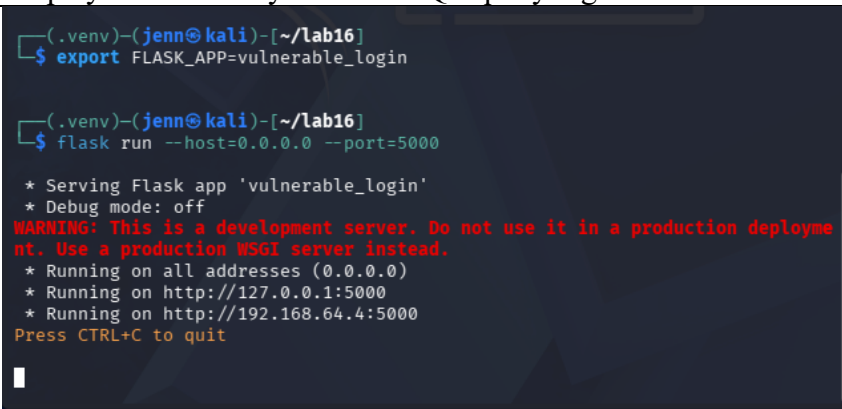
    app.run(debug=True)
```

Figure 6. vulnerable_login.py created

Displays intentionally insecure SQL query logic.

Step 7: Start the Flask server

Launches the application so the vulnerable login endpoint can be accessed and tested through a browser.

A screenshot of a terminal window titled "(.venv)-(jenn@kali)-[~/lab16]". It shows the command to export the Flask application and then run it. The output shows the server is running on port 5000, with a warning that it is a development server and should not be used in production.

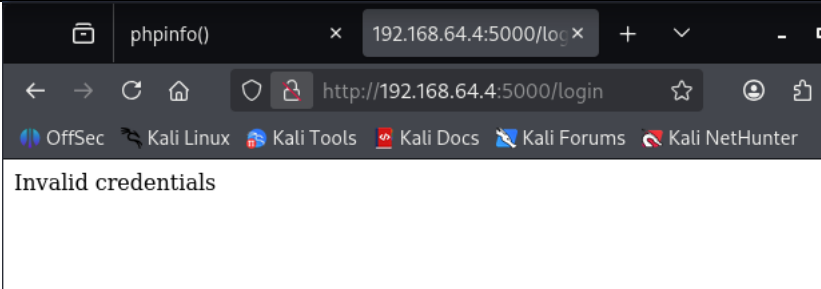
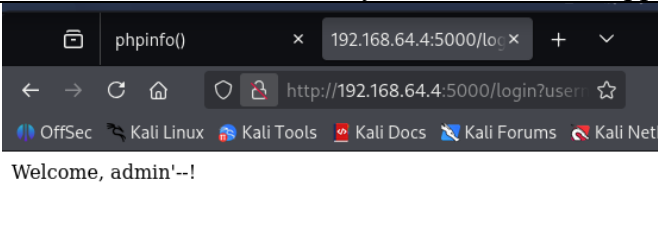
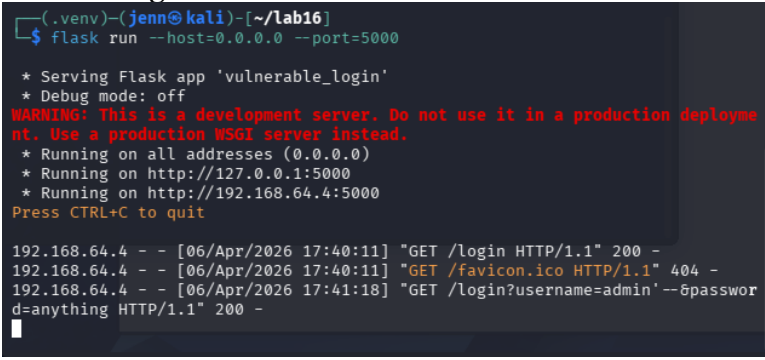
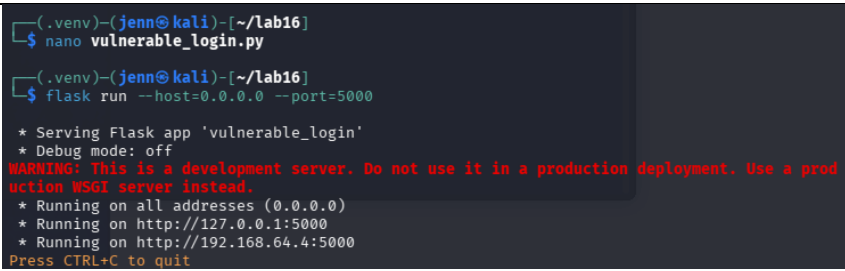
```
(.venv)-(jenn@kali)-[~/lab16]
$ export FLASK_APP=vulnerable_login

(.venv)-(jenn@kali)-[~/lab16]
$ flask run --host=0.0.0.0 --port=5000

* Serving Flask app 'vulnerable_login'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.64.4:5000
Press CTRL+C to quit
```

Figure 7. Flask server started for SQLi demo

Confirms the vulnerable app is running.

Step 8: Visit the IPv4 address to confirm the app is reachable Verifies network accessibility and confirms that Flask is correctly bound to the host interface.	 Figure 8. Browser access to login endpoint Verifies network reachability of the vulnerable app.
Step 9: Perform SQL Injection attack Executes the crafted payload to demonstrate how unvalidated input manipulates SQL structure and bypasses authentication.	 Figure 9. SQL Injection payload executed Shows authentication bypass via manipulated SQL. BONUS Figure: Results on the terminal window: 
Step 10: Fix the SQL Injection vulnerability and restart Flask Replaces unsafe string concatenation with parameterized queries, aligning with OWASP and Secure SDLC guidelines.	 Figure 10. Parameterized query fix applied Displays corrected secure SQL query implementation.

Step 11: Retest login with invalid credentials

Validates that the mitigation is effective and that the injection payload no longer bypasses authentication.

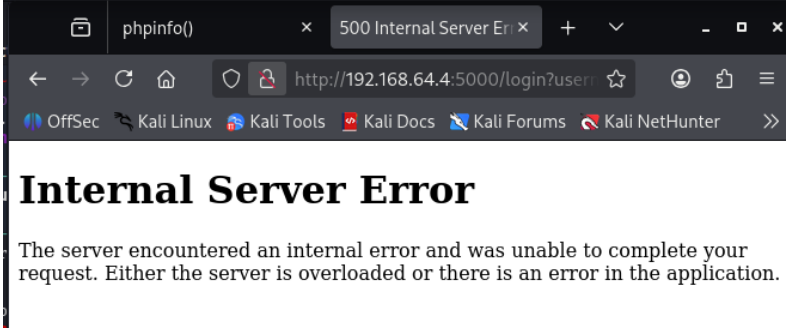


Figure 11. Failed login after fix

Confirms SQL Injection is no longer effective.

Step 12: Create the vulnerable XSS demo application

Implements an endpoint that reflects user input directly into HTML, enabling demonstration of reflected XSS.

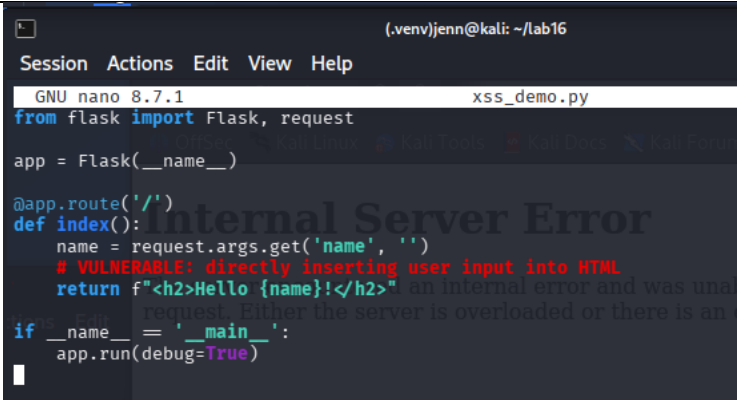


Figure 12. Vulnerable XSS app created

Shows unescaped user input in HTML output.

Step 13: Run the vulnerable XSS application

Starts the server to allow browser-based testing of unescaped user input.

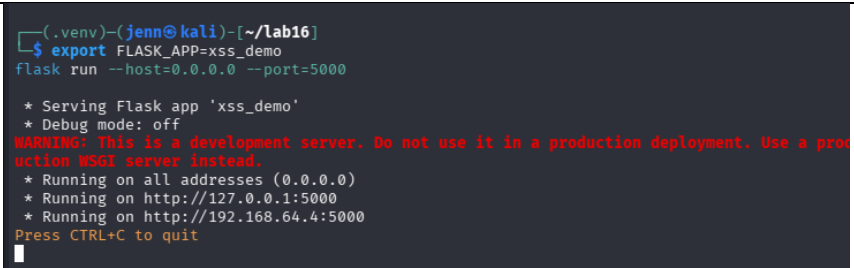
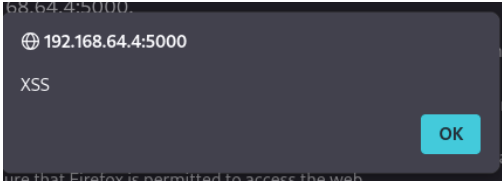
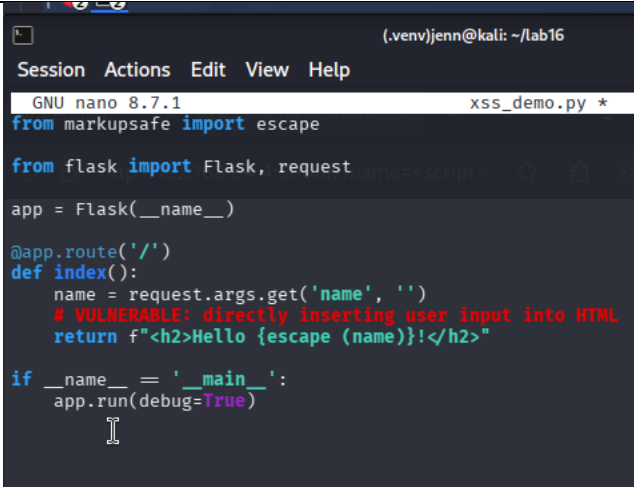
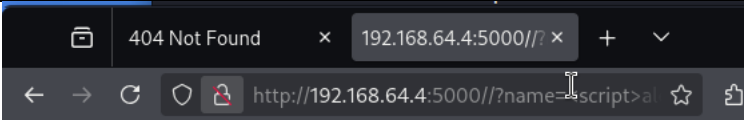


Figure 13. XSS app running

Confirms server is ready for payload testing.

Step 14: Submit XSS payload to trigger script execution Demonstrates how lack of output encoding allows attacker-controlled JavaScript to run in the victim's browser.	Payload: http://192.168.64.4:5000/?name=%3Cscript%3Ealert('XSS')%3C/script%3E Result:  Figure 14. XSS payload executed Demonstrates JavaScript execution due to missing output encoding.
Step 15: Fix xss_demo.py using markupsafe.escape() Applies proper output encoding to neutralize script tags and prevent execution.	 Figure 15. XSS fix applied using escape() Shows secure output encoding implementation via importing markupsafe and updating the “name” return field

Step 16: Retest the payload to confirm XSS is mitigated Shows that the browser now renders the payload as harmless text, validating the fix.	 Hello <script>alert('XSS')</script>! Figure 16. XSS payload neutralized Confirms script is rendered as text, not executed.
Step 17: Select and prepare a third OWASP vulnerability (Broken Access Control) Identifies an additional OWASP Top 10 category and drafts a minimal Flask app that intentionally lacks authorization checks.	This time, I checked with Google search and CoPilot for a script to test “Broken Access Control: <pre> from flask import Flask, request app = Flask(__name__) # Fake "admin only" data SECRET_DATA = "TOP SECRET: Payroll, credentials, internal keys." @app.route('/') def index(): return "<h2>Welcome! Try /admin</h2>" @app.route('/admin') def admin_panel(): # VULNERABLE: No authentication or authorization check return f"<h2>Admin Panel</h2><p>{SECRET_DATA}</p>" if __name__ == '__main__': app.run(debug=True) </pre> Figure 17. Broken Access Control script drafted Displays intentionally unprotected /admin route.

Step 18: Create broken_access.py

Implements the vulnerable /admin route that exposes sensitive data without verifying user privileges.

```
GNU nano 8.7.1 broken_access.py *
from flask import Flask, request

app = Flask(__name__)

# Fake "admin only" data
SECRET_DATA = "TOP SECRET: Payroll, credentials, internal keys."

@app.route('/')
def index():
    return "<h2>Welcome! Try /admin</h2>"

@app.route('/admin')
def admin_panel():
    # VULNERABLE: No authentication or authorization check
    return f"<h2>Admin Panel</h2><p>{SECRET_DATA}</p>"

if __name__ == '__main__':
    app.run(debug=True)
```

Figure 18. broken_access.py created

Shows vulnerable admin panel code.

Step 19: Run the broken access control application

Starts the server to demonstrate unrestricted access to the admin endpoint.

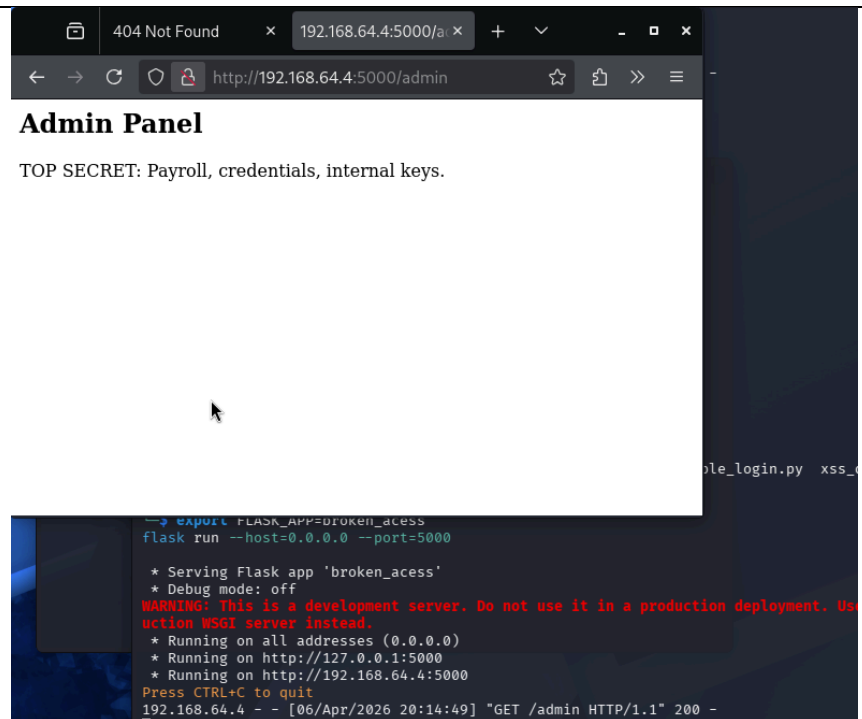
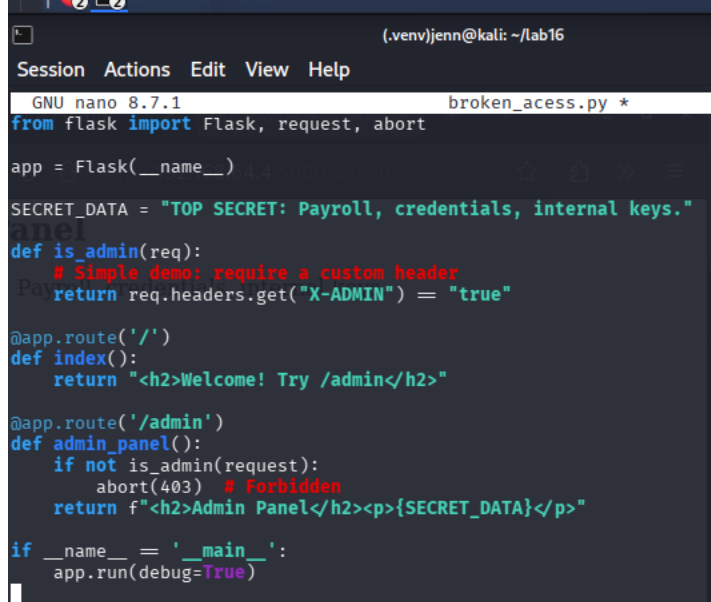


Figure 19. Broken access app running

Confirms unrestricted access to sensitive data.

Step 20: Replace vulnerable code with secure version

Adds explicit authorization logic and a 403 Forbidden response to enforce proper access control.



```
(.venv)jenn@kali: ~/lab16
Session Actions Edit View Help
GNU nano 8.7.1 broken_access.py *
from flask import Flask, request, abort

app = Flask(__name__)

SECRET_DATA = "TOP SECRET: Payroll, credentials, internal keys."

def is_admin(req):
    # Simple demo: require a custom header
    return req.headers.get("X-ADMIN") == "true"

@app.route('/')
def index():
    return "<h2>Welcome! Try /admin</h2>"

@app.route('/admin')
def admin_panel():
    if not is_admin(request):
        abort(403) # Forbidden
    return f"<h2>Admin Panel</h2><p>{SECRET_DATA}</p>"

if __name__ == '__main__':
    app.run(debug=True)
```

Figure 20. Secure version implemented

Displays authorization check and 403 logic.

Step 21: Attempt access again to validate the fix

Confirms that unauthorized users now receive a 403 error and that sensitive data is no longer exposed.

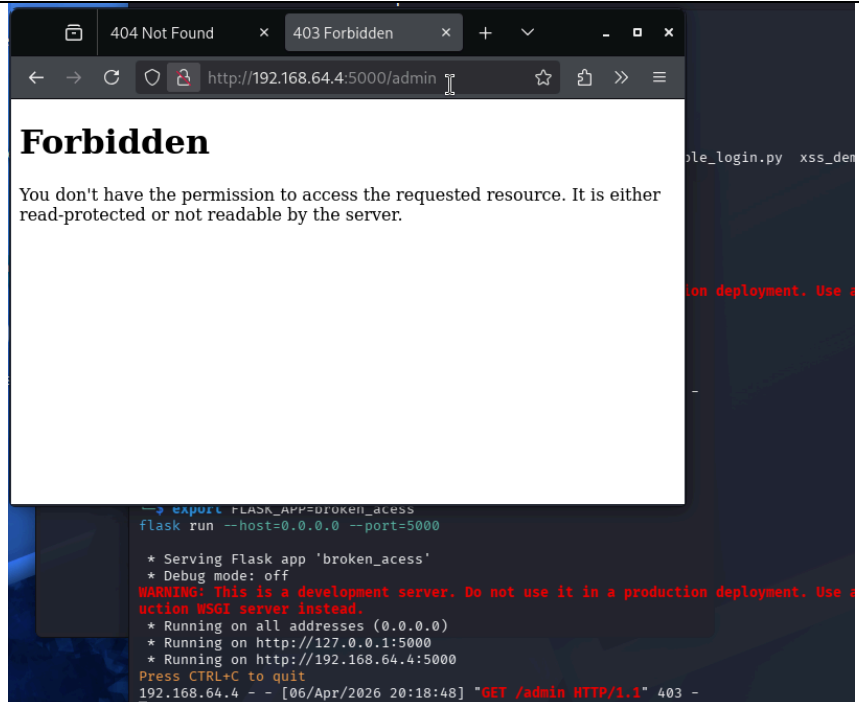


Figure 21. 403 Forbidden response after fix

Confirms unauthorized access is now blocked.

APPENDIX B: Comparison of Broken Access Script

Area	Vulnerable Version	Fixed Version
Access Control	None	Added <code>is_admin()</code> check
Authorization	Not enforced	Enforced via header
Sensitive Data Exposure	Always exposed	Protected
Error Handling	None	<code>abort(403)</code>
Imports	Flask only	Flask, request, abort
Behavior	Anyone can access <code>/admin</code>	Only authorized requests succeed